

Unit 3 — Self Study Topics

Unit: 3 | Folder: self-study | CO: CO3, CO4, CO5

This file covers the self-study topics designated for Unit 3 of the ADSA course. Each section is independent and can be studied after completing the corresponding unit lectures.

Topic 1 — Randomized Algorithms

What are Randomized Algorithms?

Algorithms that make random choices during execution. Two types:

- **Las Vegas:** Always correct; expected time is good (e.g., randomized QuickSort)
- **Monte Carlo:** May produce wrong answer with small probability; always fast (e.g., randomized primality testing)

Randomized QuickSort Analysis

```
import random

def randomized_quicksort(arr, lo, hi):
    """Expected  $O(n \log n)$ ,  $O(n^2)$  only with probability  $1/n!$ """
    if lo < hi:
        # Random pivot ensures expected  $O(n \log n)$ 
        pivot_idx = random.randint(lo, hi)
        arr[pivot_idx], arr[hi] = arr[hi], arr[pivot_idx]
        p = partition(arr, lo, hi)
        randomized_quicksort(arr, lo, p - 1)
        randomized_quicksort(arr, p + 1, hi)
```

Expected comparisons: $2n \ln n \approx 1.386 n \log_2 n$

Karger's Minimum Cut Algorithm

Repeat:

Randomly contract an edge (merge its two endpoints)

Until only 2 vertices remain

The edges between them = a cut

Probability of finding min cut: $\geq 2/(n(n-1))$

Run $n^2 \log n$ times $\rightarrow$ min cut with high probability

Skip List (Randomized Data Structure)

```
import random

class SkipNode:
```

```

def __init__(self, key, level):
    self.key = key
    self.forward = [None] * (level + 1)

class SkipList:
    """
    Expected O(log n) search, insert, delete.
    Each element independently promoted to higher levels with probabili
    """
    MAX_LEVEL = 16
    P = 0.5

    def _random_level(self):
        level = 0
        while random.random() < self.P and level < self.MAX_LEVEL:
            level += 1
        return level

```

Topic 2 — Karatsuba Algorithm

Problem

Multiply two n -digit numbers. Standard algorithm: $O(n^2)$.

Divide and Conquer Approach

For two numbers x and y , split at midpoint:

$$x = x_1 \times 10^{(n/2)} + x_0$$

$$y = y_1 \times 10^{(n/2)} + y_0$$

Naive: $x \times y = x_1y_1 \times 10^n + (x_1y_0 + x_0y_1) \times 10^{(n/2)} + x_0y_0$
 $\rightarrow$ 4 multiplications of $n/2$ -digit numbers: $T(n) = 4T(n/2)$

Karatsuba's trick: Compute $(x_1y_0 + x_0y_1)$ using only **3 multiplications**:

$$z_2 = x_1 \times y_1$$

$$z_0 = x_0 \times y_0$$

$$z_1 = (x_1 + x_0)(y_1 + y_0) - z_2 - z_0 \quad \leftarrow \text{This is } x_1y_0 + x_0y_1!$$

$$x \times y = z_2 \times 10^n + z_1 \times 10^{(n/2)} + z_0$$

```

def karatsuba(x, y):
    """
    Karatsuba multiplication.
    Time: O(n^log23) ≈ O(n^1.585) vs O(n^2) naive
    """
    if x < 10 or y < 10:
        return x * y

    # Size of numbers
    n = max(len(str(x)), len(str(y)))

```

```

m = n // 2

# Split
x1, x0 = divmod(x, 10**m)
y1, y0 = divmod(y, 10**m)

# 3 recursive multiplications (not 4!)
z2 = karatsuba(x1, y1)
z0 = karatsuba(x0, y0)
z1 = karatsuba(x1 + x0, y1 + y0) - z2 - z0

return z2 * 10**(2*m) + z1 * 10**m + z0

# Verification
a, b = 1234, 5678
print(karatsuba(a, b)) # Should be 7006652
print(a * b)          # 7006652 ✓

```

Complexity Proof

$$T(n) = 3T(n/2) + O(n)$$

By Master Theorem: $a=3$, $b=2$, $f(n)=n$, $c=\log_2 3 \approx 1.585 > 1$

$$\rightarrow T(n) = \Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$$

Applications

- Large integer arithmetic (cryptography: RSA, ECC)
- Polynomial multiplication
- Python uses Karatsuba internally for big integers

Topic 3 — Closest Pair Problem

Problem

Given n points in a 2D plane, find the pair of points with the minimum Euclidean distance.

Naive: $O(n^2)$ — check all pairs.

Divide and Conquer: $O(n \log n)$

Algorithm

```

import math

def dist(p1, p2):
    return math.sqrt((p1[0]-p2[0])**2 + (p1[1]-p2[1])**2)

def closest_pair(points):
    """Closest pair by divide and conquer.  $O(n \log n)$ """

```

```

def closest_rec(pts_x):
    n = len(pts_x)
    if n <= 3:
        # Base case: brute force
        min_d = float('inf')
        for i in range(n):
            for j in range(i+1, n):
                min_d = min(min_d, dist(pts_x[i], pts_x[j]))
        return min_d

    mid = n // 2
    mid_x = pts_x[mid][0]

    d_left = closest_rec(pts_x[:mid])
    d_right = closest_rec(pts_x[mid:])
    d = min(d_left, d_right)

    # Check strip: points within d of the dividing line
    strip = [p for p in pts_x if abs(p[0] - mid_x) < d]
    strip.sort(key=lambda p: p[1])    # Sort by y

    for i in range(len(strip)):
        j = i + 1
        while j < len(strip) and strip[j][1] - strip[i][1] < d:
            d = min(d, dist(strip[i], strip[j]))
            j += 1

    return d

pts_x = sorted(points, key=lambda p: p[0])
return closest_rec(pts_x)

```

Key Insight: Strip is $O(1)$ comparisons per point

In the strip of width $2d$, each point needs to be compared with at most **7 other points** (geometry argument: only 8 points can fit in a $d \times 2d$ rectangle with pairwise distance $\geq d$). So the strip scan is $O(n)$.

Recurrence: $T(n) = 2T(n/2) + O(n \log n) = O(n \log^2 n)$
 (With pre-sorting by y: $O(n \log n)$)

Topic 4 — Backtracking vs Divide & Conquer

Divide and Conquer

Strategy: Split problem into independent subproblems, solve, combine.

- Subproblems are independent (no shared state)
- All subproblems must be solved
- Optimal structure: optimal solution = combination of optimal subprobl

Examples: Merge Sort, Binary Search, Karatsuba, Closest Pair

Backtracking

Strategy: Build solution incrementally; abandon partial solution as soon as it cannot lead to a valid/optimal complete solution (pruning)

- Explores a tree of partial solutions
- Uses recursion + explicit undoing of choices
- Key: early pruning makes it efficient in practice

Examples: N-Queens, Sudoku, Hamiltonian Path, Subset Sum

```
def n_queens(n):
    """N-Queens using backtracking. O(n!) worst case."""
    solutions = []
    board = [-1] * n

    def is_safe(row, col):
        for r in range(row):
            if board[r] == col:
                return False
            if abs(board[r] - col) == abs(r - row):
                return False
        return True

    def solve(row):
        if row == n:
            solutions.append(board[:])
            return
        for col in range(n):
            if is_safe(row, col):
                board[row] = col      # Choose
                solve(row + 1)        # Explore
                board[row] = -1       # Undo (backtrack)

    solve(0)
    return solutions
```

Comparison Table

Feature	Divide & Conquer	Backtracking
Subproblem type	Independent, disjoint	Overlapping state
All subproblems solved	Yes	No (pruned)
Undo mechanism	Not needed	Required
Optimal guarantee	Yes (with correct design)	Yes (exhaustive)
Complexity	Usually $O(n \log n)$	$O(n!)$ to $O(2^n)$, pruning helps

Topic 5 — Greedy vs Dynamic Programming

When Greedy Works

Greedy is valid when the problem has:

1. **Greedy choice property:** Local optimal choice leads to global optimum

2. Optimal substructure: Optimal solution contains optimal solutions to subproblems

When DP is Needed

DP is needed when greedy fails — when a locally optimal choice prevents a globally optimal solution.

Classic Comparison: Coin Change

```
# Greedy works for standard denominations {1, 5, 10, 25}:
coins_greedy = lambda amount, coins: [] # Always pick largest ≤ remain

# Greedy FAILS for arbitrary denominations:
# coins = [1, 3, 4], amount = 6
# Greedy: 4+1+1 = 3 coins
# Optimal: 3+3 = 2 coins → need DP!

def coin_change_dp(coins, amount):
    """DP coin change. O(n × amount)"""
    dp = [float('inf')] * (amount + 1)
    dp[0] = 0
    for a in range(1, amount + 1):
        for c in coins:
            if c ≤ a:
                dp[a] = min(dp[a], dp[a - c] + 1)
    return dp[amount] if dp[amount] != float('inf') else -1
```

Comparison Table

Problem	Greedy	DP	Why
Activity Selection	✓	—	Exchange argument proven
Dijkstra	✓	—	Non-negative weights
Coin Change (arbitrary)	✗	✓	Local choice fails
0/1 Knapsack	✗	✓	Can't undo discrete choices
Fractional Knapsack	✓	—	Can split items
Shortest Path (neg. edges)	✗	✓	Bellman-Ford

Topic 6 — 0/1 Knapsack (Dynamic Programming)

Problem

Given n items, each with weight w_i and value v_i , and a knapsack of capacity W , select items to **maximize total value** without exceeding weight capacity. Each item is used at most once.

```
def knapsack_01(weights, values, W):
    """
    0/1 Knapsack using DP.
    Time: O(n × W) | Space: O(n × W)
    """
```

```

n = len(weights)
# dp[i][w] = max value using first i items with capacity w
dp = [[0] * (W + 1) for _ in range(n + 1)]

for i in range(1, n + 1):
    for w in range(W + 1):
        # Don't take item i
        dp[i][w] = dp[i-1][w]
        # Take item i (if it fits)
        if weights[i-1] <= w:
            dp[i][w] = max(dp[i][w],
                           dp[i-1][w - weights[i-1]] + values[i-1])

return dp[n][W]

def knapsack_trace(weights, values, W):
    """Also returns which items were selected."""
    n = len(weights)
    dp = [[0] * (W + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        for w in range(W + 1):
            dp[i][w] = dp[i-1][w]
            if weights[i-1] <= w:
                take = dp[i-1][w - weights[i-1]] + values[i-1]
                if take > dp[i][w]:
                    dp[i][w] = take

    # Backtrack to find items
    selected, w = [], W
    for i in range(n, 0, -1):
        if dp[i][w] != dp[i-1][w]:
            selected.append(i - 1)    # Item i-1 was selected
            w -= weights[i - 1]

    return dp[n][W], selected

```

Example

Items: $w=[2,3,4,5]$, $v=[3,4,5,6]$, $W=5$

dp table (items × capacity):

	0	1	2	3	4	5	
i=0	0	0	0	0	0	0	
i=1	0	0	3	3	3	3	(item 0: $w=2$, $v=3$)
i=2	0	0	3	4	4	7	(item 1: $w=3$, $v=4$) → best: take items 0+1 for
i=3	0	0	3	4	5	7	(item 2: $w=4$, $v=5$)
i=4	0	0	3	4	5	7	(item 3: $w=5$, $v=6$)

Answer: 7 (items 0 and 1)

Topic 7 — Disjoint Set (Union-Find)

Concept

A **Disjoint Set** (Union-Find) data structure maintains a collection of disjoint sets and supports:

- `find(x)` — find the representative (root) of x's set
- `union(x, y)` — merge the sets containing x and y

Implementation with Path Compression + Union by Rank

```
class UnionFind:
    """
    Union-Find with path compression and union by rank.
    find:  $O(\alpha(n))$  amortized  $\approx O(1)$  ( $\alpha$  = inverse Ackermann)
    union:  $O(\alpha(n))$  amortized
    """
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n

    def find(self, x):
        """Path compression: flatten tree during find."""
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])    # Recursive call
        return self.parent[x]

    def union(self, x, y):
        """Union by rank: attach smaller tree under larger."""
        rx, ry = self.find(x), self.find(y)
        if rx == ry:
            return False    # Already in same set

        if self.rank[rx] < self.rank[ry]:
            rx, ry = ry, rx
        self.parent[ry] = rx
        if self.rank[rx] == self.rank[ry]:
            self.rank[rx] += 1
        return True

    def connected(self, x, y):
        return self.find(x) == self.find(y)
```

Applications

```
# Kruskal's MST
def kruskal(V, edges):
    """Minimum Spanning Tree.  $O(E \log E)$ """
    edges.sort(key=lambda e: e[2])    # Sort by weight
    uf = UnionFind(V)
    mst, total = [], 0
    for u, v, w in edges:
        if uf.union(u, v):            # No cycle
```



```

        mst.append((u, v, w))
        total += w
    return mst, total

```

```

# Detecting cycles in undirected graph
# Connected components in dynamic graph
# Percolation problems

```

Topic 8 — Approximation Algorithms

When Exact Solutions Are Impractical

Many important problems are NP-hard (no known polynomial-time exact solution). Approximation algorithms find solutions guaranteed to be within a factor of the optimal.

Approximation ratio: $\text{ALG}(I) / \text{OPT}(I) \leq \rho$ for all instances I

Vertex Cover (2-Approximation)

Optimal vertex cover: NP-hard (find minimum vertex set covering all edges)

2-Approximation:

```

C = {}
while edges remain:
    pick any uncovered edge (u, v)
    add both u and v to C
    remove all edges incident to u or v

```

$|C| \leq 2 \times |\text{OPT}|$ (proof: each picked edge needs at least one endpoint)

```

def vertex_cover_2approx(V, edges):
    """2-approximation for vertex cover."""
    covered = set()
    cover = set()
    for u, v in edges:
        if u not in covered and v not in covered:
            cover.add(u)
            cover.add(v)
            covered.add(u)
            covered.add(v)
    return cover

```

Traveling Salesman Problem (1.5-Approximation — Christofides)

For **metric TSP** (satisfies triangle inequality):

1. Find minimum spanning tree T
2. Find minimum weight perfect matching M on odd-degree vertices of T
3. Combine $T + M \rightarrow$ Eulerian multigraph
4. Find Euler tour $\rightarrow$ shortcut to Hamiltonian cycle

Ratio: 1.5 (proven; cannot be improved assuming $P \neq NP$)

Greedy Set Cover ($\ln n$ approximation)

```
def set_cover_greedy(universe, sets):
    """ln n approximation for Set Cover.  $O(n \times |sets|)$ """
    covered, selected = set(), []
    while covered != universe:
        # Pick the set covering the most uncovered elements
        best = max(sets, key=lambda s: len(s - covered))
        selected.append(best)
        covered |= best
    return selected
```

Topic 9 — Branch and Bound

Concept

Branch and Bound is an exact algorithm for optimization problems that prunes the search space using a **bound function**. Unlike backtracking (feasibility), branch and bound tracks the **best solution** found so far.

```
Maintain: best_solution = best found so far
For each partial solution:
    Compute upper bound (or lower bound for minimization)
    If bound ≤ best_solution: prune this branch (cannot improve)
    Else: branch (explore further)
```

0/1 Knapsack via Branch and Bound

```
import heapq

def knapsack_branch_bound(weights, values, W):
    """
    Branch and bound for 0/1 knapsack.
    Better than DP for sparse high-value solutions.
    """
    n = len(weights)
    # Sort by value/weight ratio (greedy bound)
    items = sorted(zip(values, weights), key=lambda x: x[0]/x[1], reverse=True)

    def upper_bound(level, profit, weight):
        """Fractional knapsack bound (overestimate of remaining profit)"""
        bound = profit
        w = weight
        for i in range(level, n):
            v, wt = items[i]
            if w + wt <= W:
                bound += v; w += wt
            else:
                bound += v * (W - w) / wt
```

```

        break
    return bound

best = [0]
# Priority queue: (-bound, level, profit, weight)
heap = [(-upper_bound(0, 0, 0), 0, 0, 0)]

while heap:
    neg_bound, level, profit, weight = heapq.heappop(heap)
    bound = -neg_bound

    if bound <= best[0]:
        continue # Prune
    if level == n:
        best[0] = max(best[0], profit)
        continue

    v, w = items[level]

    # Branch: take item
    if weight + w <= W:
        new_profit = profit + v
        best[0] = max(best[0], new_profit)
        ub = upper_bound(level + 1, new_profit, weight + w)
        if ub > best[0]:
            heapq.heappush(heap, (-ub, level + 1, new_profit, weight))

    # Branch: skip item
    ub = upper_bound(level + 1, profit, weight)
    if ub > best[0]:
        heapq.heappush(heap, (-ub, level + 1, profit, weight))

return best[0]

```

Comparison: Backtracking vs Branch & Bound

Feature	Backtracking	Branch & Bound
Goal	Find any valid solution	Find optimal solution
Pruning condition	Infeasibility	Suboptimality (bound)
Bound function	Not needed	Required (quality of pruning)
Used for	Constraint satisfaction	Optimization (TSP, knapsack)
Complexity	$O(b^d)$ pruned	$O(b^d)$ pruned (better pruning)

References

- Cormen et al., *Introduction to Algorithms*, 4th Ed. (MIT Press, 2022) — Ch. 4 (D&C), Ch. 15 (DP), Ch. 16 (Greedy), Ch. 35 (Approximation)
- Skiena, *The Algorithm Design Manual*, 3rd Ed. (Springer, 2020) — Ch. 9 (Intractable Problems), Ch. 11 (NP-Hard Approximation)
- Laaksonen, *Guide to Competitive Programming*, 3rd Ed. (Springer, 2024) — Ch. 13

(Dynamic Programming)

- Halim et al., *Competitive Programming*, 4th Ed. (Lulu, 2020)